

3

# 栈和队列

理论课程



廈門大學  
XIAMEN UNIVERSITY



信息学院 黃 煒  
(特色化示范性软件学院) 博士, 副教授  
School of Informatics Wei Huang

# 内容要点

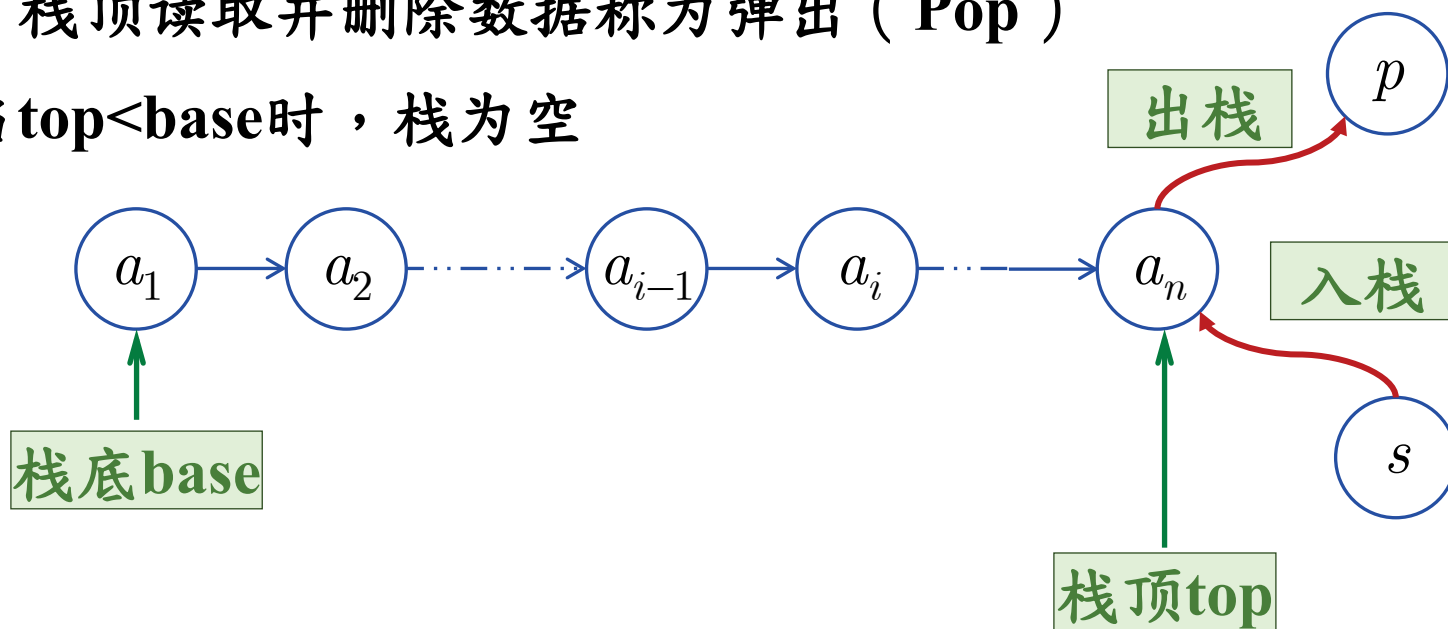
- 栈
  - 顺序栈
  - 链栈
  - 栈的应用
- 队列
  - 顺序队列
  - 链队列
  - 队列的应用

# 目录

|  |     |      |
|--|-----|------|
|  | 1   | 栈    |
|  | 1.1 | 基本概念 |
|  | 1.2 | 顺序栈  |
|  | 2   | 队列   |
|  | 3   | 小结   |

# 基本概念

- 栈和队列都是限制在端点进行插入和删除的线性表。
- 栈是一种后进先出(LIFO)的线性表。
  - 插入数据至栈顶称为压入 ( Push )
  - 自栈顶读取并删除数据称为弹出 ( Pop )
  - 当 $\text{top} < \text{base}$ 时，栈为空



**例2-6** 一个栈的输入序列为  $a_1 a_2 a_3 a_4 a_5$  ,  
则栈的输出序列不可能是( )。

(A)  $a_2 a_3 a_4 a_1 a_5$

(B)  $a_2 a_3 a_1 a_4 a_5$

(C)  $a_5 a_4 a_1 a_3 a_2$

(D)  $a_1 a_5 a_4 a_3 a_2$

# 栈的抽象数据类型定义

- 定义

ADT Stack{

数据对象： $D = \{ a_i \mid a_i \in \text{ElemSet}, i=1,2,\dots,n, n \geq 0 \}$

数据关系：

$R = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i=2,3,\dots,n \}$

基本操作：

初始化、进栈、出栈、取栈顶元素、  
判断栈空等

} ADT Stack

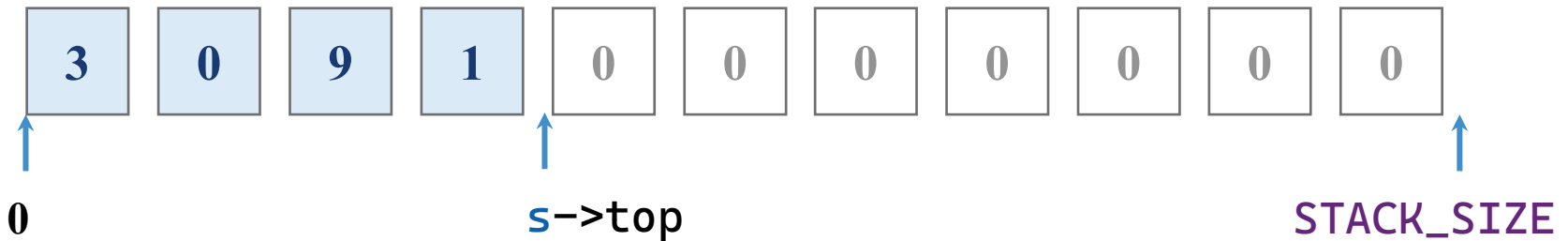
# 目录

|  |     |      |
|--|-----|------|
|  | 1   | 栈    |
|  | 1.1 | 基本概念 |
|  | 1.2 | 顺序栈  |
|  | 1.3 | 链栈   |
|  | 2   | 队列   |

# 顺序栈的存储结构

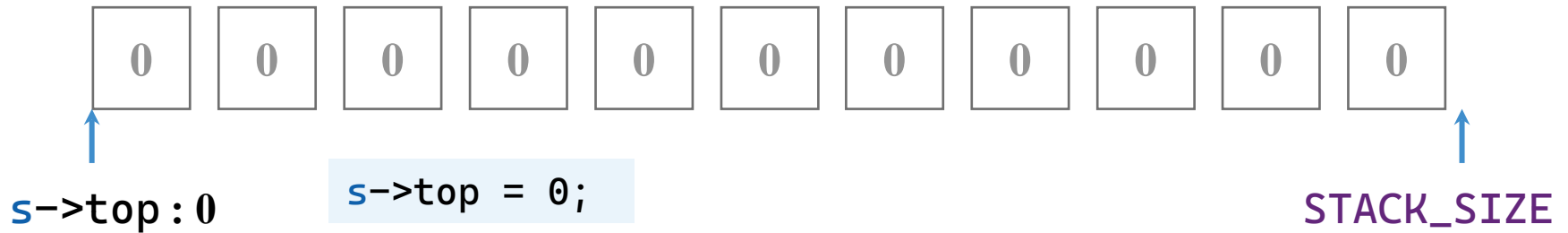
- 顺序栈的存储结构

```
typedef struct {  
    int data[STACK_SIZE];  
    int top;  
} SeqStack;
```



# 顺序栈：初始化

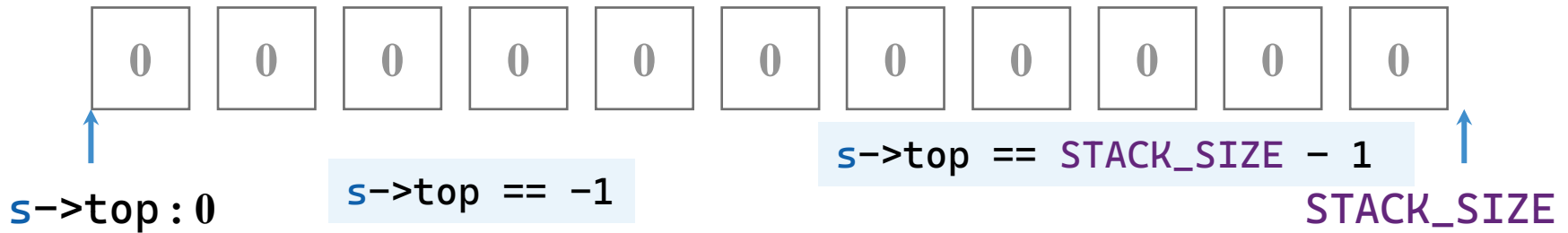
- 初始化：长度清零（栈顶为-1）



```
int init_stack(SeqStack* s) {  
    if (s == NULL)  
        return ERROR;  
    s->top = -1;  
    return OK;  
}
```

# 顺序栈：判断空和满

- 判断空：栈顶是否小于0
- 判断满：栈顶是否达到栈长度最大值减一



```
int is_empty(SeqStack* s) {  
    if (s == NULL)  
        return ERROR;  
    return (s->top == -1);  
}
```

```
int is_full(SeqStack* s) {  
    if (s == NULL)  
        return ERROR;  
    return (s->top == STACK_SIZE - 1);  
}
```

# 顺序栈：压入栈

- 入栈：赋值到顶部并前进

主体步骤



```
int push(SeqStack* s, int x) {  
    if (s == NULL || is_full(s))  
        return ERROR;  
    s->top++;  
    s->data[s->top] = x;  
    return OK;  
}
```

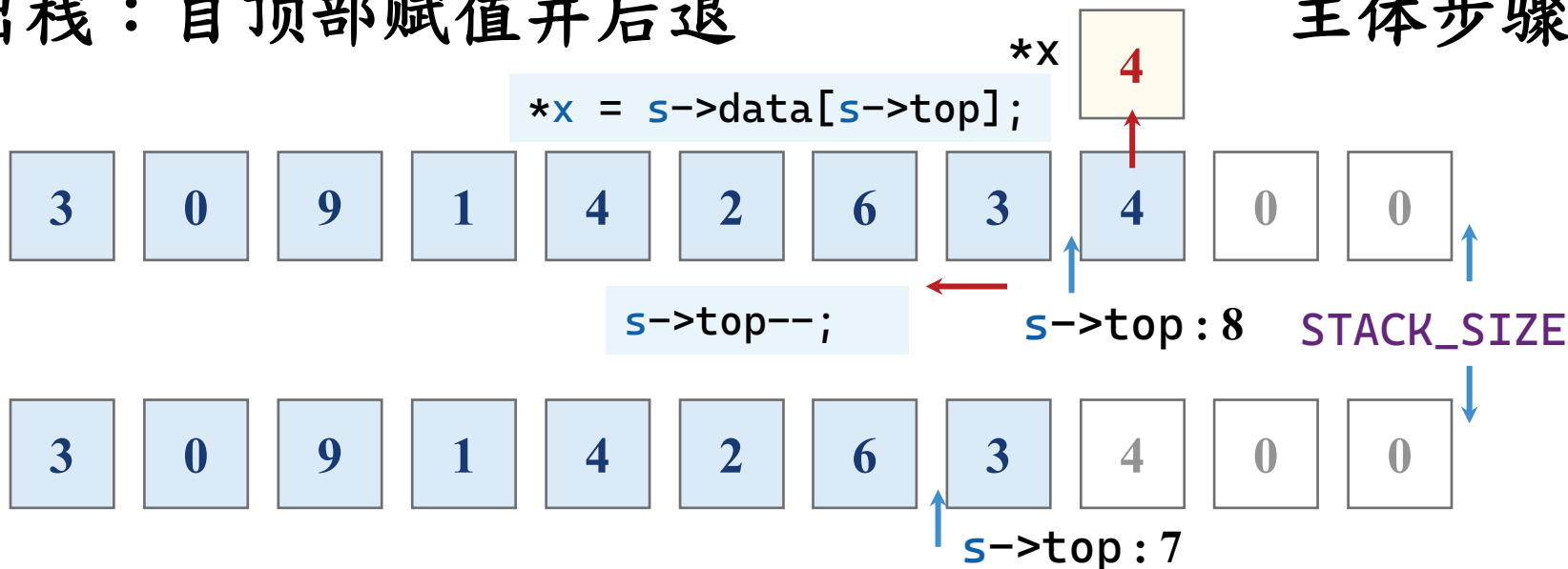
边界判定

1. 栈指针空
2. 栈满

# 顺序栈：弹出栈

- 出栈：自顶部赋值并后退

主体步骤



```
int pop(SeqStack* s, int* x) {  
    if (s == NULL || is_empty(s) || x == NULL)  
        return ERROR;  
    *x = s->data[s->top];  
    s->top--;  
    return OK;  
}
```

边界判定

1. 栈指针空
2. 栈空
3. 出指针空

# 顺序栈：取栈顶

- 出栈：自顶部赋值并不动

主体步骤



```
int get_top(SeqStack* s, int* x) {  
    if (s == NULL || is_empty(s) || x == NULL)  
        return ERROR;  
    *x = s->data[s->top];  
    return OK;  
}
```

边界判定

1. 栈指针空
2. 栈空
3. 出指针空

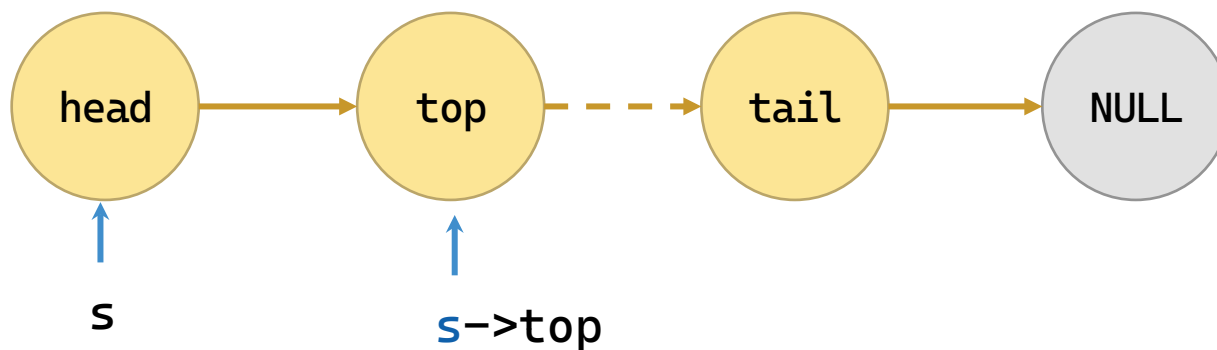
# 目录

|  |     |      |
|--|-----|------|
|  | 1   | 栈    |
|  | 1.2 | 顺序栈  |
|  | 1.3 | 链栈   |
|  | 1.4 | 栈的应用 |
|  | 2   | 队列   |

# 链栈的存储结构

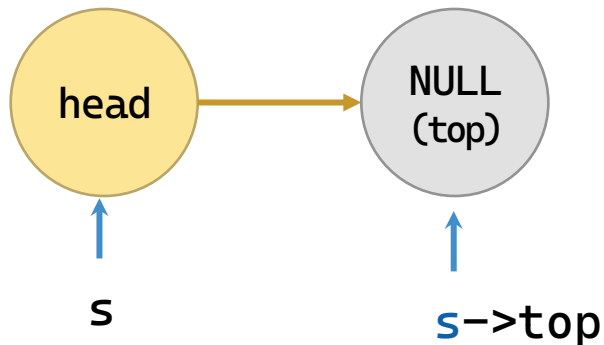
- 顺序栈的存储结构

```
typedef struct LinkNode {  
    int data;  
    struct LinkNode* next;  
} LinkNode;  
typedef LinkNode* LinkStack;
```



# 链栈：初始化

- 初始化：栈的头结点之后为空



```
LinkStack init_stack(void) {  
    LinkNode* head = (LinkNode*)malloc(sizeof(LinkNode));  
    if (head != NULL) {  
        head->data = 0;           // 头结点数据域未使用  
        head->next = NULL;  
    }  
    return head;                 // 返回头结点指针，失败则返回 NULL  
}
```

# 链栈：判断空

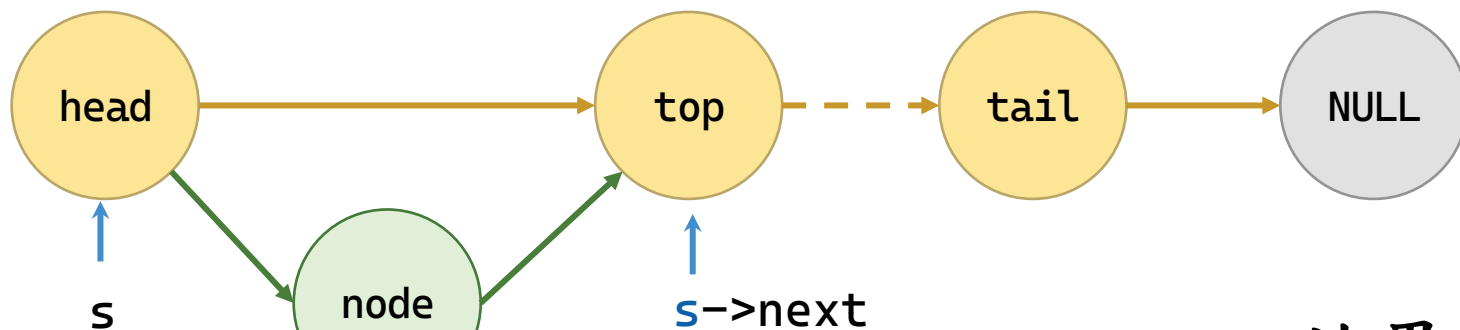
- 判断空：栈的头结点之后是否为空
- 判断满：内部没有办法申请新的空间即是满

```
int is_empty(LinkStack stack) {  
    if (stack == NULL)  
        return ERROR;           // 栈未初始化  
    return (stack->next == NULL);  
}
```

# 链表：压入栈

## • 入栈：头插法

## 主体步骤

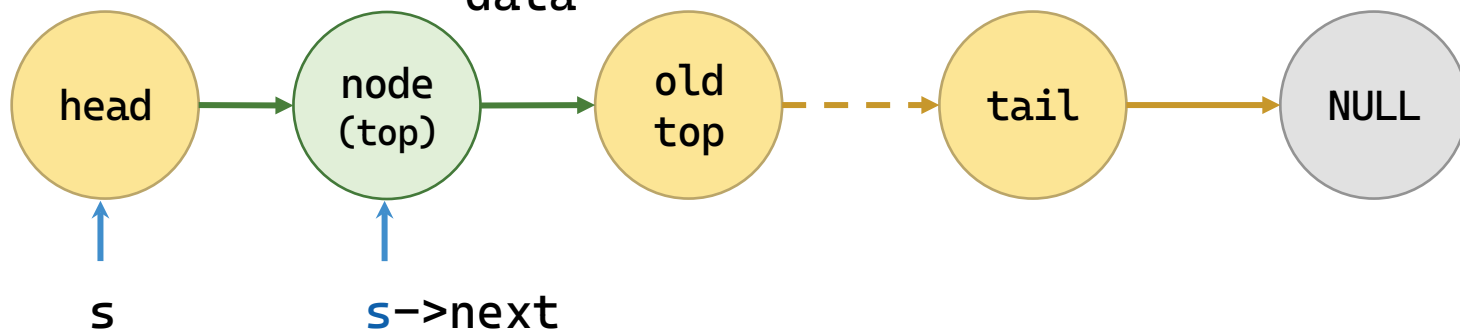


`stack->next = node;`

`node->data = data;`  
`node->next = stack->next;`

## 边界判定

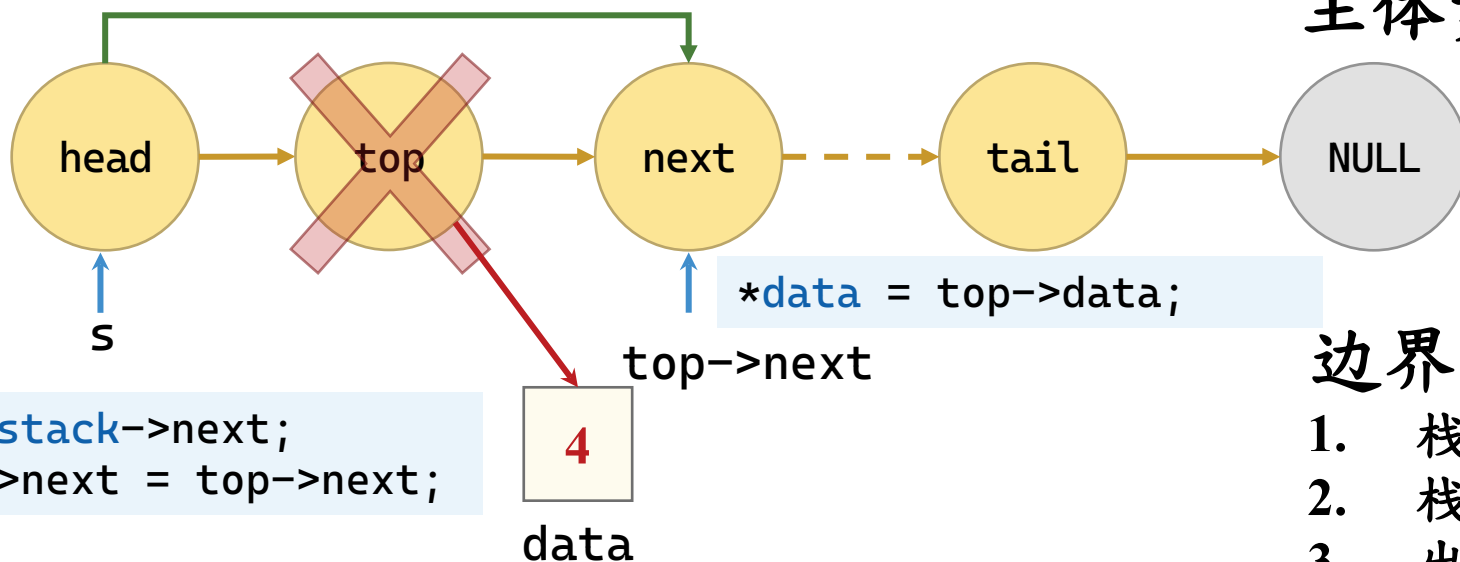
1. 栈指针空
2. 栈满



```
int push(LinkStack stack, int data) {  
    if (stack == NULL)  
        return ERROR;  
    LinkNode* node = (LinkNode*)malloc(sizeof(LinkNode));  
    if (node == NULL)  
        return ERROR;  
    node->data = data;  
    node->next = stack->next;  
    stack->next = node;  
    return OK;  
}
```

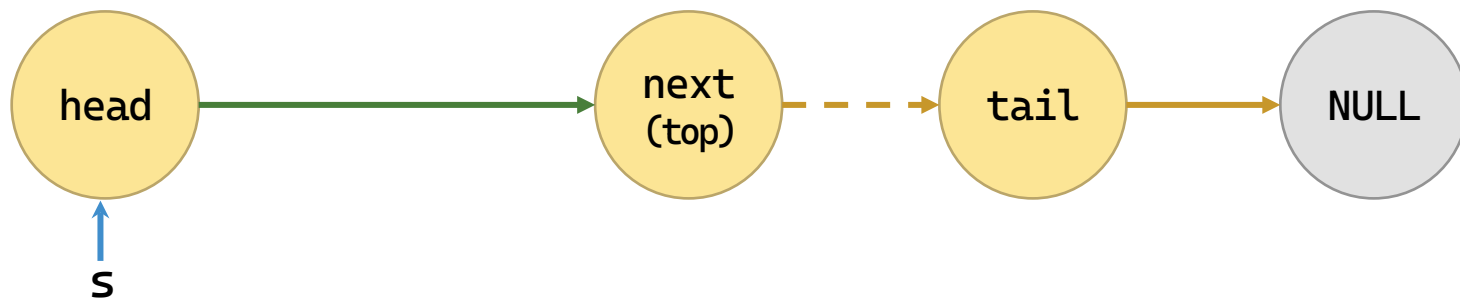
# 链表：弹出栈和取栈顶

- 出栈：取出并删除栈顶结点；取栈顶：取值后不动
- 主体步骤



## 边界判定

1. 栈指针空
2. 栈空
3. 出指针空



```
int pop(LinkStack stack, int* data) {  
    if (stack == NULL || is_empty(stack) || data == NULL)  
        return ERROR;  
    LinkNode* top = stack->next;  
    *data = top->data;  
    stack->next = top->next;  
    free(top);  
    return OK;  
}
```

```
int get_top(LinkStack stack, int* data) {  
    if (stack == NULL || is_empty(stack) || data == NULL)  
        return ERROR;  
    LinkNode* top = stack->next;  
    *data = top->data;  
    return OK;  
}
```

# 目录

|  |     |      |
|--|-----|------|
|  | 1   | 栈    |
|  | 1.2 | 顺序栈  |
|  | 1.3 | 链栈   |
|  | 1.4 | 栈的应用 |
|  | 2   | 队列   |

# 栈应用：进制转换

- 例2-8 输入10进制数n，输出d进制数。

– 原理： $n = (n \text{ div } d) \times d + n \text{ mod } d$

- 试算： $(1348)_{10} = (2504)_8$ ，运算过程：

$$1348 \div 8 = 168 \dots\dots 4$$

$$168 \div 8 = 21 \dots\dots 0$$

$$21 \div 8 = 2 \dots\dots 5$$

$$2 \div 8 = 0 \dots\dots 2$$

- 逆序输出

```
int temp = n;
do {
    push(stack, temp % d);
    temp /= d;
} while (temp != 0);
```

```
while (!is_empty(stack)) {
    pop(stack, &digit);
    if (digit < 10)
        printf("%d", digit);
    else
        printf("%c", 'A'+digit-10);
}
```

```

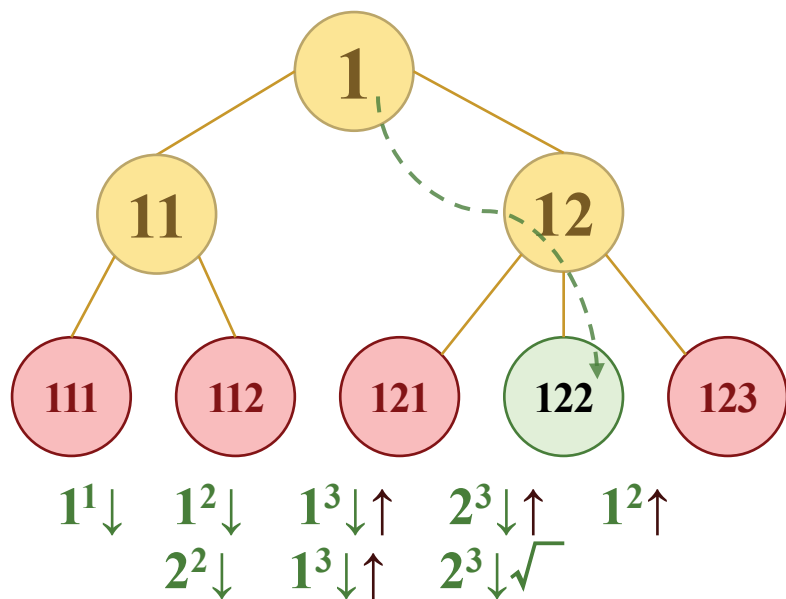
void convert_base(void) {
    int n, d, digit;
    LinkStack stack = init_stack();    // 创建临时栈
    if (stack == NULL) {
        printf("栈初始化失败! \n");
        return;
    }
    printf("请输入十进制整数 n: "); scanf("%d", &n);
    printf("请输入目标进制 d (2~16) : "); scanf("%d", &d);
    if (d < 2 || d > 16) {
        printf("进制输入无效, 请输入2~16之间的整数。 \n");
        free(stack);
        return;
    }
    int temp = n;
    do {
        push(stack, temp % d);
        temp /= d;
    } while (temp != 0);
    printf("十进制 %d 转换为 %d 进制为: ", n, d);
}

```

```
while (!is_empty(stack)) {  
    pop(stack, &digit);  
    if (digit < 10)  
        printf("%d", digit);  
    else  
        printf("%c", 'A' + digit - 10);  
}  
printf("\n");  
free(stack);  
}
```

# 栈应用：迷宫

- 例2-9 假设迷宫由m行n列构成，有一个入口(1, 1)和一个出口(m, n)。试找出一条从入口通往出口的可行路径（如：输出可通行路径方格的坐标序列）。
- 思路：深度优先搜索



|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |   |
|---|---|---|---|---|---|---|---|---|---|
| 1 | → | → |   |   |   | * |   |   | 1 |
| 2 |   | ↓ | → | → |   | * |   |   | 2 |
| 3 | * | * | ← | ↓ | * |   |   |   | 3 |
| 4 |   | ← | ↓ | * | ↑ | → | → |   | 4 |
| 5 |   | ↓ | * |   | ↑ | * | ↓ | * | 5 |
| 6 |   | ↓ | * | ↑ | → | * | ↓ | → | 6 |
| 7 |   | ↓ | → | → | * | * |   | ↓ | 7 |
|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |   |

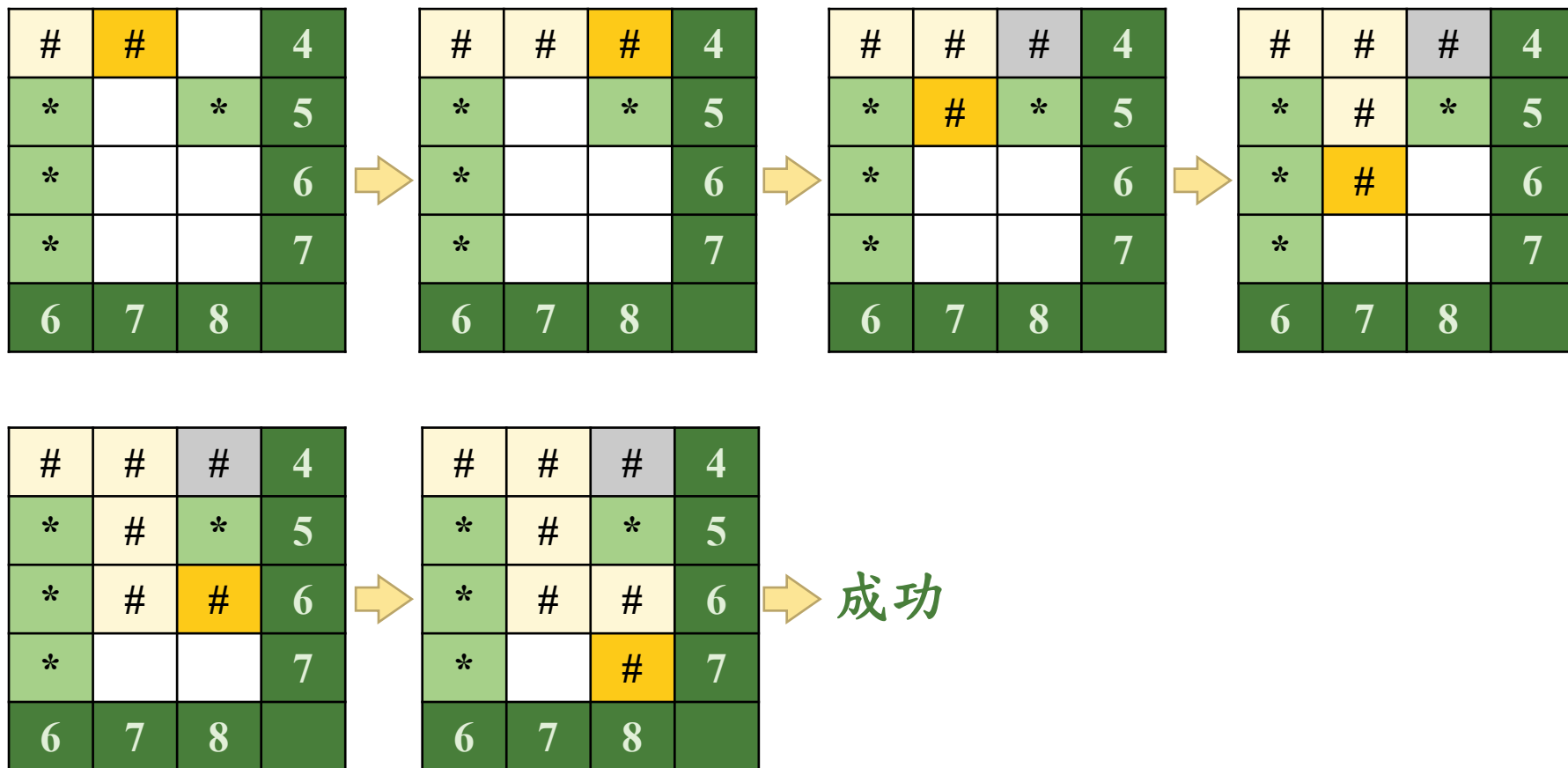
# 栈应用：迷宫

- 思路：

- 从入口开始。
- 如果当前位置有通行路径，纳入可行路径（进栈），按照右→下→左→上的顺序依次选择1个方向前进；
- 如果当前位置的三个前进方向均不可通行，后退，并将当前位置从可行路径中删去（出栈）；
- 如果当前位置为出口位置，搜索成功；
- 如果当前位置为入口位置，且没有可通行路径，搜索失败。

# 栈应用：迷宫

- 思路：数据结构：栈；算法：DFS



# 栈应用：迷宫

- 主体部分：按四个方向走一步，走即入栈
  - 下一步：判断已走过和走不通

```
int dirs[4][2] = { {0,1}, {1,0}, {0,-1}, {-1,0} };
push(stack, 0);
while (!is_empty(stack)) {
    int pos;
    get_top(stack, &pos);
    int x = pos % cols, y = pos / cols; // 列, 行
    for (int d = 0; d < 4; d++) {
        int nx = x + dirs[d][0], ny = y + dirs[d][1];
        if (nx >= 0 && nx < cols && ny >= 0 && ny < rows) {
            int npos = ny * cols + nx;
            push(stack, npos);
        }
    }
}
```

# 栈应用：迷宫

- 补充：已走过则标记上，走不通就是没动过

```
int moved = 0;
for (int d = 0; d < 4; d++) {
    int nx = x + dirs[d][0], ny = y + dirs[d][1];
    if (nx >= 0 && nx < cols && ny >= 0 && ny < rows) {
        int npos = ny * cols + nx;
        if (maze[npos] == 0 && !visited[npos]) {
            push(stack, npos);
            visited[npos] = 1;
            moved = 1;
            break;
        }
    }
}
if (!moved) { // 无路可走，回退
    int tmp;
    pop(stack, &tmp);
}
```

# 栈应用：迷宫

- 记录找到了，找到则输出（应该逆序输出）

```
if (x == cols - 1 && y == rows - 1) { // 到达出口
    found = 1;
    break;
}
```

```
if (found) {
    printf("找到路径: \n");
    while (!is_empty(stack)) {
        int p;
        pop(stack, &p);
        printf("(%d, %d) ", p % cols, p / cols);
    }
    printf("\n");
    free(rev);
} else
    printf("无法找到路径! \n");
```

```

void maze_path(void) {
    int rows, cols;
    printf("请输入迷宫行数和列数: ");
    scanf("%d %d", &rows, &cols);
    // 一维数组存储迷宫, 下标 = y*cols + x, calloc初始化为0
    int* maze = (int*)malloc(rows * cols * sizeof(int));
    int* visited = (int*)calloc(rows * cols, sizeof(int));
    if (maze == NULL || visited == NULL) {
        printf("内存分配失败! \n");
        free(maze); free(visited);
        return;
    }
    printf("请输入迷宫矩阵 (0通路, 1墙) : \n");
    for (int i = 0; i < rows * cols; i++)
        scanf("%d", &maze[i]); // 直接按一维顺序读入
    // 方向: 右、下、左、上 (列增量, 行增量)
    int dirs[4][2] = { {0,1}, {1,0}, {0,-1}, {-1,0} };
    LinkStack stack = init_stack();
    if (stack == NULL) {
        printf("栈初始化失败! \n");
        free(maze); free(visited);
        return;
    }
}

```

```

push(stack, 0); visited[0] = 1; // 起点 (0,0), 0*cols+0=0
int found = 0;
while (!is_empty(stack)) {
    int pos; get_top(stack, &pos);
    int x = pos % cols, y = pos / cols; // 列, 行
    if (x == cols - 1 && y == rows - 1) { // 到达出口
        found = 1; break;
    }
    int moved = 0;
    for (int d = 0; d < 4; d++) {
        int nx = x + dirs[d][0], ny = y + dirs[d][1];
        if (nx >= 0 && nx < cols && ny >= 0 && ny < rows) {
            int npos = ny * cols + nx;
            if (maze[npos] == 0 && !visited[npos]) {
                push(stack, npos);
                visited[npos] = 1;
                moved = 1;
                break;
            }
        }
    }
}
if (!moved) { // 无路可走, 回退
    int tmp; pop(stack, &tmp);
}
}

```

```

if (found) {
    printf("找到路径: \n");
    // 逆序输出 (用辅助栈)
    LinkStack rev = init_stack();
    while (!is_empty(stack)) {
        int p;
        pop(stack, &p);
        push(rev, p);
    }
    while (!is_empty(rev)) {
        int p; pop(rev, &p);
        printf("(%d, %d) ", p % cols, p / cols);
    }
    printf("\n");
    while (!is_empty(rev)) {
        int tmp; pop(rev, &tmp);
    }
    free(rev);
} else
    printf("无法找到路径! \n");
free(stack);
free(maze);
free(visited);
}

```

# 应用：括号匹配检验

- 例2-10（括号匹配检验） 设在表达式中允许包含[]和()两种括号，约定

- ([ ] ( ) ) 或 [ ( [ ] [ ] ) ] 这类括号匹配格式是正确的

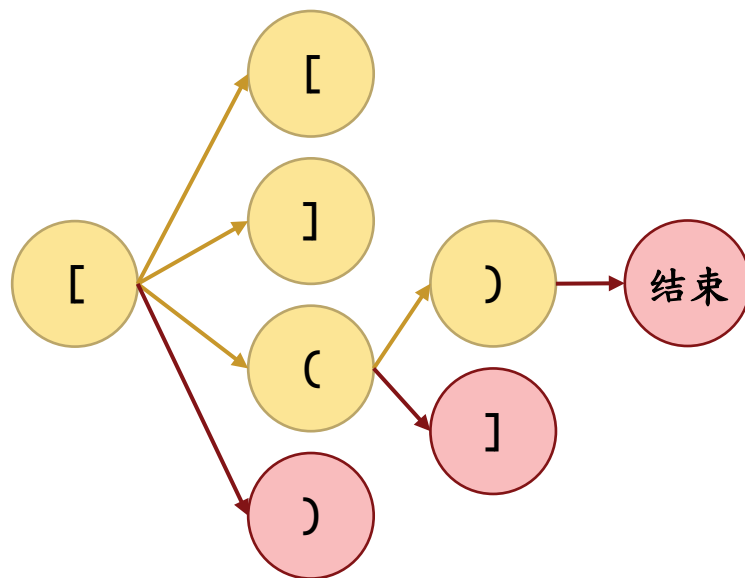
- [ ( ] ) 或 ( [ ( ) ) 或 ( ( ) ] 这类括号匹配格式是不正确的。

- 思路

- 左 括 号：压栈

- 右 括 号：匹配则出栈  
不匹配则失败

- 输入结束：非空栈则失败



# 应用：括号匹配检验

## • 核心代码

```
for (i = 0; expr[i] != '\0'; i++) {
    char ch = expr[i];
    if (ch == '(' || ch == '[' || ch == '{') {
        push(stack, ch);
    } else if (ch == ')' || ch == ']' || ch == '}') {
        if (is_empty(stack))
            break;
        int left; get_top(stack, &left);
        if ((ch == ')' && left == '(') || (ch == ']' && left
== '[') || (ch == '}' && left == '{')) {
            int tmp; pop(stack, &tmp);
        } else
            break;
    }
}
if (expr[i] == '\0' && is_empty(stack))
    printf("括号匹配正确! \n");
else
    printf("括号匹配错误! \n");
```

```

void bracket_match(void) {
    char expr[256];
    printf("请输入包含括号的表达式: ");
    scanf("%s", expr);
    LinkStack stack = init_stack();
    if (stack == NULL) {
        printf("栈初始化失败! \n");
        return;
    }
    int i = 0;
    for (i = 0; expr[i] != '\0'; i++) {
        char ch = expr[i];
        if (ch == '(' || ch == '[' || ch == '{') {
            push(stack, ch);    // 将字符 ASCII 码压栈
        } else if (ch == ')' || ch == ']' || ch == '}') {
            if (is_empty(stack))
                break;
            int topChar;
            get_top(stack, &topChar);
            char left = (char)topChar;

```

```

        if ((ch == ')' && left == '(') ||
            (ch == ']' && left == '[') ||
            (ch == '}' && left == '{')) {
            int tmp;
            pop(stack, &tmp);
        } else
            break;
    } // 忽略其他字符
}
if (expr[i] == '\0' && is_empty(stack)) {
    printf("括号匹配正确! \n");
} else {
    printf("括号匹配错误! \n");
}
while (!is_empty(stack)) { // 释放栈
    int tmp;
    pop(stack, &tmp);
}
free(stack);
}

```

# 应用：算术表达式求值

- 例2-11 （ 算术表达式求值 ） 设有
  - 符号集： $\{ 0, 1, \dots, 9, ., +, -, *, /, (, ), \# \}$
  - 运算规则（ $\#$ 为表达式的结束标识）：
    - (1)先计算最内层括号内的表达式；
    - (2)先 $*$ 、 $/$ ，后 $+$ 、 $-$ ；
    - (3)从左至右计算 $*$ 和 $/$ 或者 $+$ 和 $-$ 。
- 思路
  - 先把中缀表达式转换为后缀表达式
  - 对后缀表达式求值

# 应用：算术表达式求值

- 步骤1：先把中缀表达式转换为后缀表达式

式： $10+(18+9*3)/15-6 \rightarrow 10, 18, 9, 3, *, +, 15, /, 6, -$

| 读 | 处理 | 栈    | 后缀表达式         |
|---|----|------|---------------|
| 1 | 存  |      | 1             |
| 0 | 存  |      | 10,           |
| + | 入  | +    | 10,           |
| ( | 入  | +(   | 10,           |
| 1 | 存  | +(   | 10, 1         |
| 8 | 存  | +(   | 10, 18,       |
| + | 存  | +(+  | 10, 18,       |
| 9 | 存  | +(+  | 10, 18, 9,    |
| * | 入  | +(+* | 10, 18, 9,    |
| 3 | 存  | +(+* | 10, 18, 9, 3, |

| 读 | 处理 | 栈  | 后缀表达式                            |
|---|----|----|----------------------------------|
| b | 出  | s  | 10, 18, 9, 3, *,                 |
|   | 出  | +( | 10, 18, 9, 3, *, +,              |
|   | 出  | +  | 10, 18, 9, 3, *, +,              |
| / | 入  | +/ | 10, 18, 9, 3, *, +,              |
| 1 | 存  | +/ | 10, 18, 9, 3, *, +, 1            |
| 5 | 存  | +/ | 10, 18, 9, 3, *, +, 15,          |
| - | 出  | +  | 10, 18, 9, 3, *, +, 15, /,       |
|   | 出  |    | 10, 18, 9, 3, *, +, 15, /, +,    |
|   | 入  | -  | 10, 18, 9, 3, *, +, 15, /, +,    |
| 6 | 存  | -  | 10, 18, 9, 3, *, +, 15, /, +, 6, |

# 应用：算术表达式求值

- 步骤1：先把中缀表达式转换为后缀表达式

式： $10+(18+9*3)/15-6 \rightarrow 10, 18, 9, 3, *, +, 15, /, 6, -$

— 规矩：数字入串，优先级低则入栈，高则出栈入串

- 遇到数字，则存入后缀串
- 遇到左括号，则入栈
- 遇到运算符且栈顶优先级更低，则入栈
- 遇到右括号，则出栈，存入后缀串
- 遇到运算符且栈顶优先级更高，则出栈，存入后缀串
- 遇到输入结束，则出栈，存入后缀串

```

static int priority(char op) {
    switch (op) {
        case '+': case '-': return 1;
        case '*': case '/': return 2;
        default: return 0;
    }
}

void infix_to_postfix(const char* infix, char* postfix) {
    LinkStack ops = init_stack();
    if (!ops) return;
    int i = 0, j = 0;
    while (infix[i] != '#' && infix[i] != '\0') {
        char ch = infix[i];
        if (ch >= '0' && ch <= '9') {
            while (ch >= '0' && ch <= '9') {
                postfix[j++] = ch;
                i++;
                ch = infix[i];
            }
            postfix[j++] = ' ';
            continue;
        }
    }
}

```

```

} else if (ch == '(') {
    push(ops, ch);
} else if (ch == ')') {
    int top;
    get_top(ops, &top);
    while ((char)top != '(') {
        postfix[j++] = (char)top;
        postfix[j++] = ' ';
        pop(ops, &top);
        get_top(ops, &top);
    }
    pop(ops, &top);    // 弹出 '('
} else if (ch == '+' || ch == '-' || ch == '*' || ch == '/') {
    int top;
    get_top(ops, &top);
    while (!is_empty(ops) && (char)top != '('
        && priority((char)top) >= priority(ch)) {
        postfix[j++] = (char)top;
        postfix[j++] = ' ';
        pop(ops, &top);
        get_top(ops, &top);
    }
}

```

```

        }
        push(ops, ch);
    }
    i++;
}
while (!is_empty(ops)) {
    int top;
    pop(ops, &top);
    postfix[j++] = (char)top;
    postfix[j++] = ' ';
}
postfix[j] = '\0';
while (!is_empty(ops)) {
    int tmp; pop(ops, &tmp);
}
free(ops);
}

```

# 应用：算术表达式求值

## • 步骤2：后缀表达式求值

式：10, 18, 9, 3, \*, +, 15, /, +, 6, -

式：10, 18, 27, +, 15, /, +, 6, -

式：10, 45, 15, /, +, 6, -

式：10, 3, +, 6, -

式：13, 6, -

式：7

— 规矩：

- 遇到数字，则入栈
- 遇到运算符，则两次出栈，得到的两个运算数与运算符进行运算

```

int eval_postfix(const char* postfix) {
    LinkStack nums = init_stack();
    if (!nums) return ERROR;
    int i = 0, error = 0;
    while (postfix[i] != '\0' && !error) {
        if (postfix[i] >= '0' && postfix[i] <= '9') {
            int num = 0;
            while (postfix[i] >= '0' && postfix[i] <= '9') {
                num = num * 10 + (postfix[i] - '0');
                i++;
            }
            push(nums, num);
        } else if (postfix[i] == '+' || postfix[i] == '-'
            || postfix[i] == '*' || postfix[i] == '/') {
            int b, a, res;
            pop(nums, &b);
            pop(nums, &a);
            switch (postfix[i]) {
                case '+': res = a + b; break;
                case '-': res = a - b; break;
                case '*': res = a * b; break;
                case '/':
                    if (b == 0) {
                        printf("错误: 除零! \n"); error = 1; res = 0;
                    }
            }
        }
        i++;
    }
    return res;
}

```

```

        } else
            res = a / b;
        break;
    }
    if (!error) push(nums, res);
}
i++;
}
int result;
if (!error && get_top(nums, &result) == OK) {
    while (!is_empty(nums)) {
        int tmp; pop(nums, &tmp);
    }
    free(nums);
    return result;
} else {
    while (!is_empty(nums)) {
        int tmp; pop(nums, &tmp);
    }
    free(nums);
    return ERROR;
}
}

```

# 应用：递归

- 递归函数：一个利用自身定义的函数。
- 递归算法
  - 将待求解问题 $F(n)$ 进行分解
  - 转化为求解相对简单的子问题，如 $F(n-1)$
  - 其中一些子问题比较容易得到解，如 $F(1)$ ，称为递归出口。
  - 而其它子问题可以用同样方法进行求解，但问题规模较小。
- 一个递归算法应具有下列特点
  - (1)递归出口至少一个。否则，算法将永远不会有结果
  - (2)在经过有限次的递归调用后，能够导致递归出口的出现。

# 应用：递归（Ackerman函数）

- 例1-12 Ackerman函数。它是一种双递归函数。

$$\text{Ack}(m, n) = \begin{cases} n+1, & m=0 \\ \text{Ack}(m-1, 1), & m>0 \text{ 且 } n=0 \\ \text{Ack}(m-1, \text{Ack}(m, n-1)), & m>0 \text{ 且 } n>0 \end{cases}$$

```
int ack(int m, int n) {  
    if (m == 0) {  
        return n + 1;  
    } else if (n == 0) {  
        return ack(m - 1, 1);  
    } else {  
        return ack(m - 1, ack(m, n - 1));  
    }  
}
```

# 应用：递归（Ackerman函数）

## • 代码分析

- 栈溢出风险：Ackerman函数的递归深度极深。
  - 仅仅计算  $A(4, 1)$  就会产生极其庞大的递归调用树，这会导致程序直接因为内存耗尽（栈溢出）而崩溃。建议  $m$  的值不要超过 3。
- 时间复杂度：计算时间随着  $m$  和  $n$  的增加呈爆炸式增长。
  - $A(3, 2)$  的计算已经需要几十次递归调用，而  $A(3, 3)$  的结果是 61，但计算过程却极其漫长。
- 记忆化优化：计算过程中会产生大量重复的子问题，可以通过引入二维数组作为缓存表来存储已经计算过的结果，从而大幅减少重复计算的时间。

# 递归的栈实现

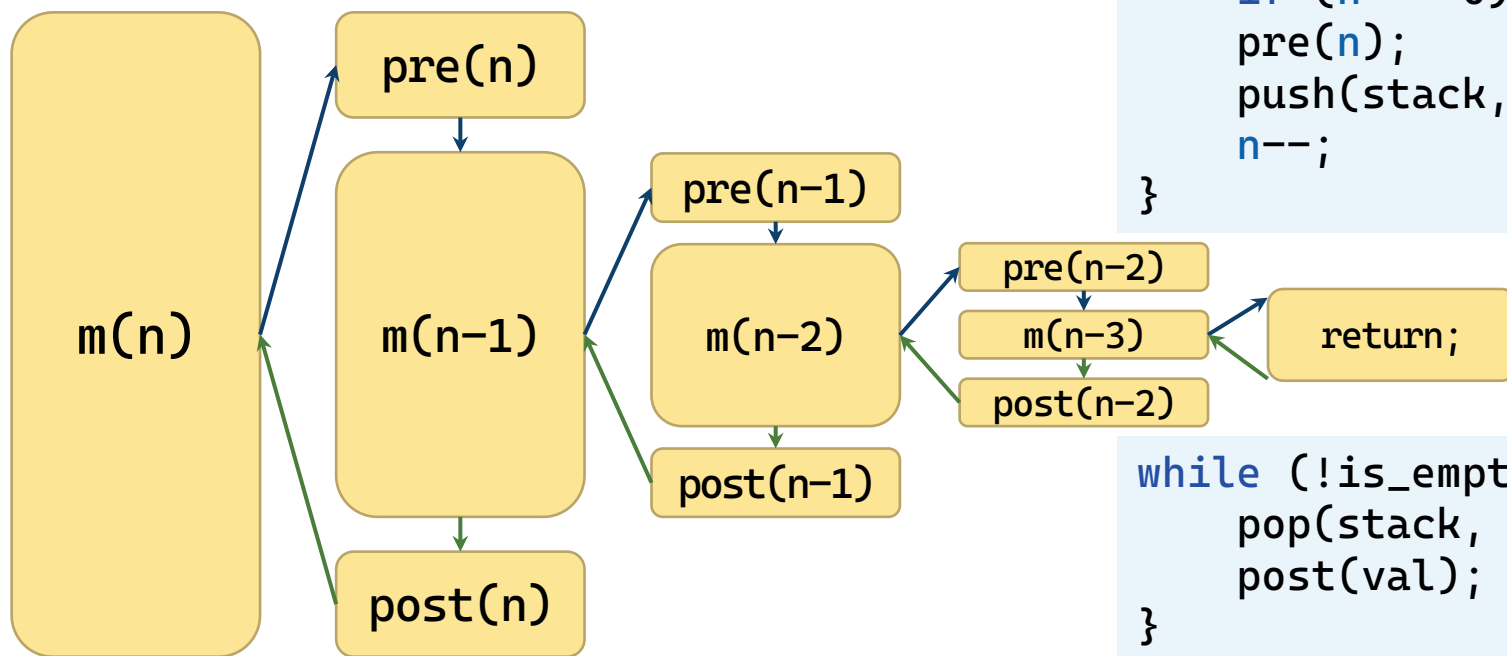
- 进入递归时，系统的操作
  - 保存当前状态入栈：参数值、局部变量、当前执行位置
  - 进入新一轮函数调用
- 退出递归时，系统的操作
  - 获取返回值作为被调用函数的值
  - 恢复栈顶的状态，继续执行主调用函数
- 问题：系统维护的栈通常只有1MB大小
- 如果需要栈超过系统限制，需要自己改用栈实现

# 递归的栈实现

## • 原递归

```
void m(int n) {
    if (n == 0) return; // 出口
    pre(n);
    m(n - 1);
    post(n);
}
```

```
while (1) {
    if (n == 0) break; //出口
    pre(n);
    push(stack, n);
    n--;
}
```



```
while (!is_empty(stack)) {
    pop(stack, &val);
    post(val);
}
```

```

void iterative_m(int n) {
    LinkStack stack = init_stack();
    if (!stack) return;
    // 阶段1: 模拟递归进入
    while (1) {
        if (n == 0) break;    // 递归出口, 退出进入阶段
        pre(n);
        push(stack, n);
        n--;
    }
    // 阶段2: 模拟递归返回
    int val;
    while (!is_empty(stack)) {
        pop(stack, &val);
        post(val);
    }
    free(stack);
}

```

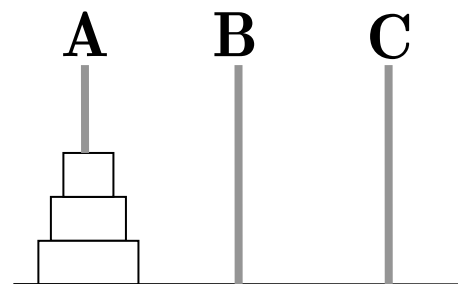
```

int ackermann_with_stack(int m, int n) {
    LinkStack stack = init_stack();
    if (stack == NULL) return ERROR; // 栈初始化失败
    while (m > 0) {
        if (push(stack, m) == ERROR) return ERROR;
        m--;
    }
    int result = n + 1; // A(0, n) = n + 1
    int cur_m;
    while (!is_empty(stack)) { // 不断处理栈中的 m
        if (pop(stack, &cur_m) == ERROR) return ERROR;
        if (cur_m == 1) {
            result = result + 1;
        }
        else {
            if (push(stack, cur_m-1) == ERROR) return ERROR;
            result = 1;
        }
    }
    return result;
}

```

# 应用：Hanoi塔

- 例1-13 Hanoi塔问题：已知有3个塔座A, B, C和 $n$ 个圆盘（直径 $d_i$ 满足 $d_1 < d_2 < \dots < d_n$ ）。假设已经从小到大依次将这些圆盘叠放在塔座x上。要求按照下列规则，将塔座x上的 $n$ 个圆盘都移动到塔座z上：
  - (1) 一次只能移动一个圆盘；
  - (2) 圆盘可以临时叠放在x, y或z塔座上；
  - (3) 任一次移动后，任一塔座上都不能出现大圆盘在小圆盘之上的现象。



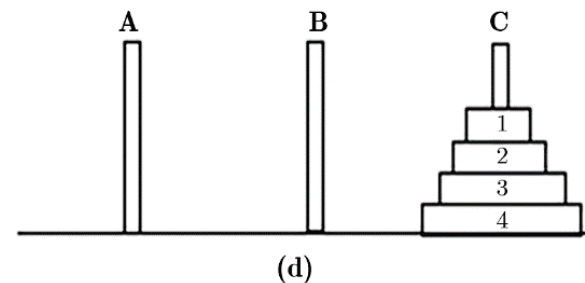
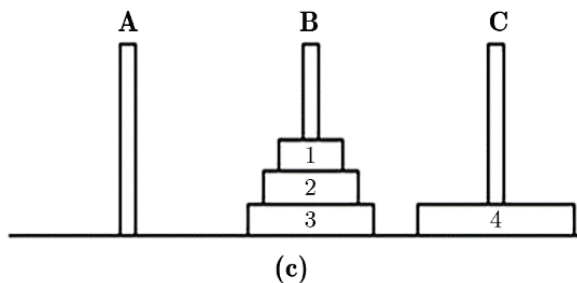
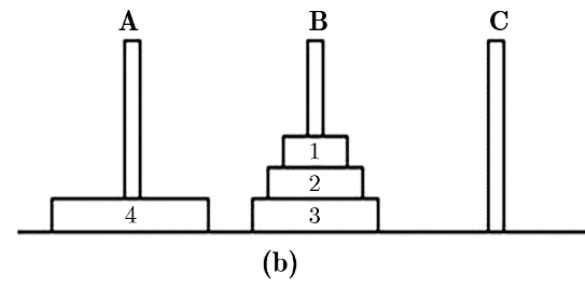
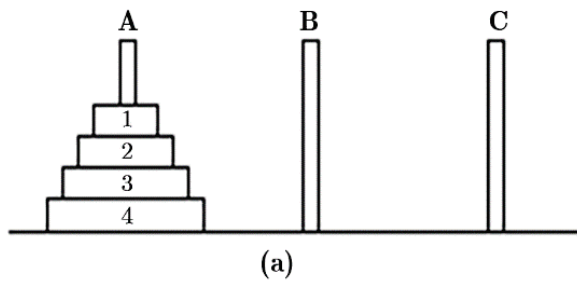
# 应用：Hanoi塔

## • 汉诺塔

–  $1 \sim n-1$  :  $A \rightarrow B$

–  $n$  :  $A \rightarrow C$

–  $1 \sim n-1$  :  $B \rightarrow C$



```
void hanoi(int n, char from, char to, char aux) {  
    if (n == 0) return;  
    hanoi(n - 1, from, aux, to);  
    move_count++;  
    printf("%d: %c --> %c\n", n, from, to);  
    hanoi(n - 1, aux, to, from);  
}
```

```

void hanoi_stack(int n, char from, char to, char aux) {
    LinkStack stack = init_stack();
    if (!stack) {
        printf("栈初始化失败! \n");
        return;
    }
    push(stack, (HanoiParam) { n, from, to, aux, 0 }); // 初始
    while (!is_empty(stack)) {
        HanoiParam f;
        pop(stack, &f);
        if (f.n == 0) continue; // 出口
        if (f.stage == 0) {
            f.stage = 1; // 当前帧 stage 设为 1
            push(stack, f); // 重新压栈一遍激发第二个递归
            push(stack, (HanoiParam) { f.n - 1, f.from, f.aux,
                f.to, 0 }); // 第1递归 hanoi(n-1, from, aux, to)
        } else { // stage == 1
            printf("%d: %c --> %c\n", f.n, f.from, f.to);
            push(stack, (HanoiParam) { f.n - 1, f.aux, f.to,
                f.from, 0 }); // 第2个递归 hanoi(n-1, aux, to, from)
        }
    }
    free(stack); // 释放头结点
}

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
/* 返回值状态码 */
#define ERROR    -1
#define OK       1
typedef struct {
    int n;           // 盘子数
    char from;       // 源柱
    char to;         // 目标柱
    char aux;        // 辅助柱
    int stage;       // 0: 第一个递归前; 1: 第二个递归前
} HanoiParam;
typedef struct LinkNode {
    HanoiParam data;
    struct LinkNode* next;
} LinkNode;          // 链式栈结点
typedef LinkNode* LinkStack; // 栈类型: 指向头结点的指针
int is_empty(LinkStack stack) {
    if (stack == NULL) return ERROR;           // 栈未初始化
    return (stack->next == NULL);
}

```

```

LinkStack init_stack(void) {
    LinkNode* head = (LinkNode*)malloc(sizeof(LinkNode));
    if (head != NULL)
        memset(head, 0, sizeof(*head));
    return head;           // 返回头结点指针, 失败则返回 NULL
}

int push(LinkStack stack, HanoiParam data) {
    if (stack == NULL) return ERROR;           // 栈未初始化
    LinkNode* newNode = (LinkNode*)malloc(sizeof(LinkNode));
    if (newNode == NULL) return ERROR;         // 内存分配失败
    newNode->data = data;
    newNode->next = stack->next;               // 新结点指向原栈顶
    stack->next = newNode;                     // 头结点指向新栈顶
    return OK;
}

int pop(LinkStack stack, HanoiParam* data) {
    if (stack == NULL) return ERROR;
    LinkNode* topNode = stack->next;           // 获取栈顶结点
    if (topNode == NULL) return ERROR;         // 栈空
    *data = topNode->data;                     // 返回数据
    stack->next = topNode->next;               // 头结点跳过栈顶
    free(topNode);                            // 释放原栈顶
}

```

```

    return OK;
}
int get_top(LinkStack stack, HanoiParam* data) {
    if (stack == NULL) return ERROR;
    LinkNode* topNode = stack->next;
    if (topNode == NULL) return ERROR;
    *data = topNode->data;
    return OK;
}
int main() {
    int n;
    printf("请输入汉诺塔的盘子数量: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("输入无效, 请输入正整数! \n");
        return 1;
    }
    printf("汉诺塔 %d 层移动步骤: \n", n);
    hanoi_stack(n, 'A', 'C', 'B');
    return 0;
}

```

# 应用：背包问题

- 例1-14（背包问题） 设有  $n$  个物品，每个物品的重量为  $w_i$ ,  $i=1, 2, \dots, n$ 。试从这  $n$  个物品中选取若干个，使其重量之和 = 背包的容量  $T$ 。
- 例：  $n=5$ ，  $W=\{17, 51, 28, 32, 63\}$ ，  $T=100$ 。
- 选择  $w_1 + w_2 + w_4 = 17 + 51 + 32 = 100$
- 思路：DFS
  - 如果  $S \subseteq W = \{w_i\}$  是  $(W, T)$  的可行解
  - 对于任何  $s_i \in S$ ，  $S \setminus \{s_i\}$  也是  $(W \setminus \{s_i\}, T - s_i)$  可行解
  - 对于任何  $s_i \notin S$ ，  $S \cup \{s_i\}$  也是  $(W \cup \{s_i\}, T + s_i)$  可行解

```

#include <stdio.h>
#include <stdlib.h>
#define MAX_N 100
int w[MAX_N];           // 物品重量数组
int selected[MAX_N];    // 标记选中物品 (1选中, 0未选)
int n, T;               // 物品个数, 背包容量

int knap_rec(int i, int sum) {
    if (sum == T) return 1;    // 出口, 找到一组可行解
    if (i == n || sum > T) return 0;    // 无解
    selected[i] = 1; // 尝试选第 i 个物品
    if (knap_rec(i + 1, sum + w[i]))
        return 1;    // 有选可行, 则传到至上层
    selected[i] = 0; // 尝试“不选”第 i 个物品
    if (knap_rec(i + 1, sum))
        return 1;    // 不选可行, 则传到至上层
    return 0;        // 都不可行
}

```

```

int main() {
    printf("请输入物品个数 n: "); scanf("%d", &n);
    printf("请输入背包容量 T: "); scanf("%d", &T);
    printf("请输入 %d 个物品的重量: ", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &w[i]);
    if (knap_rec(0, 0)) {
        printf("找到一组解: ");
        for (int i = 0; i < n; i++) {
            if (selected[i])
                printf("%d ", w[i]);
        }
        printf("= %d\n", T);
    } else {
        printf("无解! \n");
    }
    return 0;
}

```

```

typedef struct {
    int i;           // 当前物品下标
    int sum;         // 当前已选重量
    int stage;       // 0: 刚进入, 准备尝试选; 1: 选已尝试, 准备尝试不选
} Param;

typedef struct LinkNode {
    Param data;
    struct LinkNode* next;
} LinkNode;

typedef LinkNode* LinkStack;

#define MAX_N 100
int w[MAX_N];           // 物品重量数组
int selected[MAX_N];    // 标记选中物品 (1选中, 0未选)
int n, T;               // 物品个数, 背包容量

int knap_stack(void) {
    LinkStack stack = init_stack();
    if (stack == NULL) {
        printf("栈初始化失败! \n");
        return 0;
    }
    push(stack, (Param) { 0, 0, 0 });
}

```

```

while (!is_empty(stack)) {
    Param f; pop(stack, &f);
    if (f.sum == T) { // 递归出口: 找到解
        while (!is_empty(stack)) {
            Param tmp; pop(stack, &tmp);
        }
        free(stack);
        return 1;
    }
    if (f.i == n || f.sum > T)
        continue; // 无解: 物品用完或超重
    if (f.stage == 0) { // 已在待选阶段
        // 压栈, 以备稍后处理“不选”
        push(stack, (Param) { f.i, f.sum, 1 });
        selected[f.i] = 1; // 标记选中
        push(stack, (Param) { f.i + 1, f.sum + w[f.i], 0 });
    } else { // 尝试“不选”当前物品
        selected[f.i] = 0; // 标记不选
        push(stack, (Param) { f.i + 1, f.sum, 0 });
    }
}
free(stack);
return 0; // 无解
}

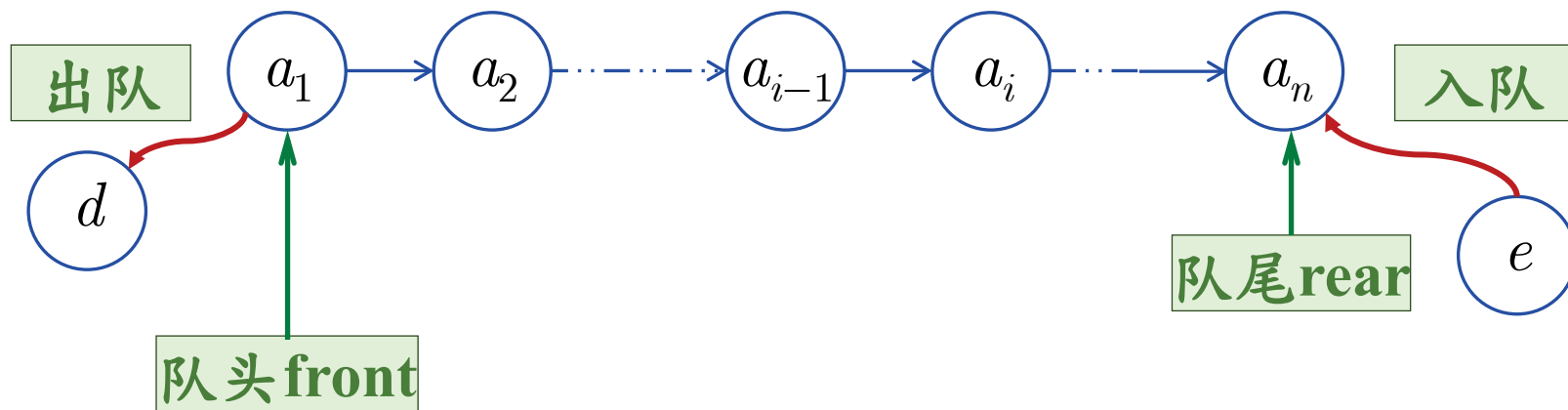
```

# 目录

|  |     |      |
|--|-----|------|
|  | 1   | 栈    |
|  | 2   | 队列   |
|  | 2.1 | 基本概念 |
|  | 2.2 | 顺序队列 |
|  | 2.3 | 链队列  |

# 基本概念

- 栈和队列都是限制在端点进行插入和删除的线性表。
- 队列是一种先进先出(FIFO)的线性表。
  - 插入数据至队列尾部顶称为入队 ( Enqueue )
  - 自队列头部读取并删除数据称为出队 ( Dequeue )
  - 当 $\text{front}=\text{rear}$ 时，队为空



# 队列的抽象数据类型定义

- 定义

ADT Queue {

数据对象： $D = \{ a_i \mid a_i \in \text{ElemSet}, i=1,2,\dots,n, n \geq 0 \}$

数据关系：

$R = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i=2,3,\dots,n \}$

约定  $a_1$  端为队头， $a_n$  端为队尾。

基本操作：

初始化、进队、出队、判断队空等

} ADT Queue

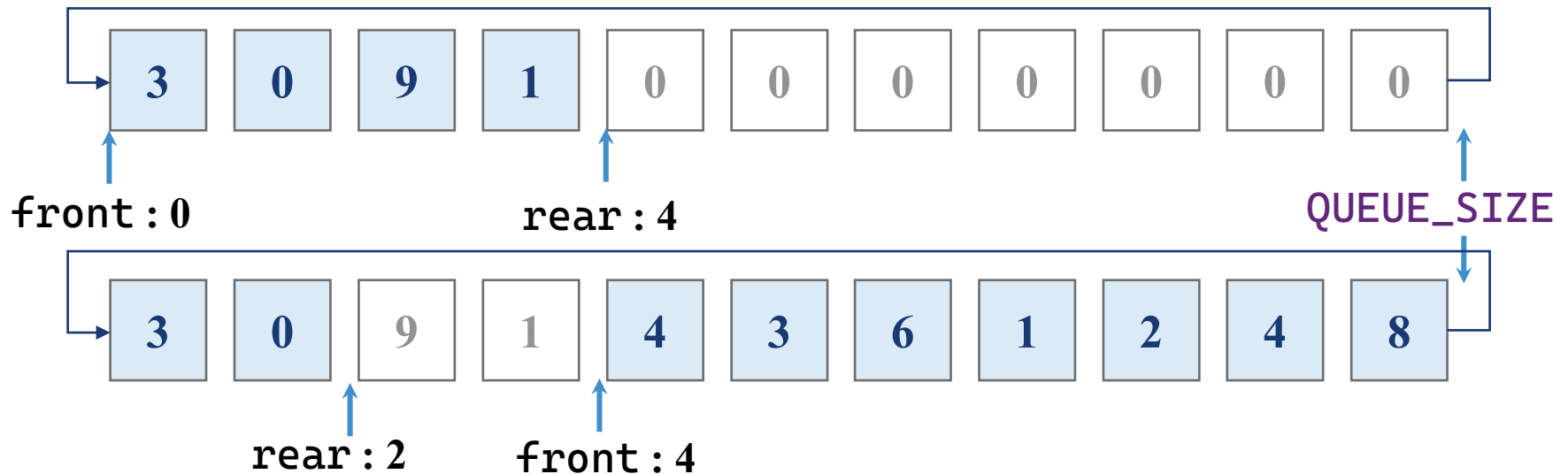
# 目录

|  |     |      |
|--|-----|------|
|  | 1   | 栈    |
|  | 2   | 队列   |
|  | 2.1 | 基本概念 |
|  | 2.2 | 顺序队列 |
|  | 2.3 | 链队列  |

# 顺序队列的存储结构

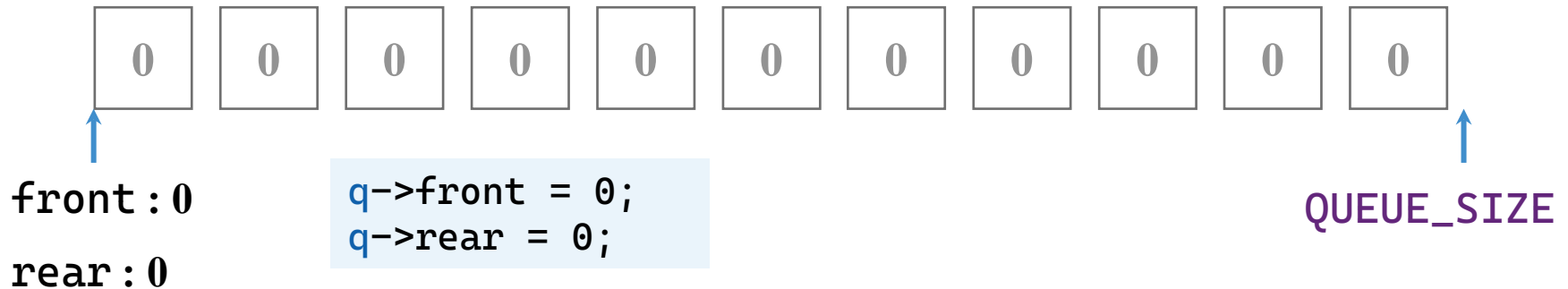
- 顺序队列（本节为循环版本）的存储结构

```
typedef struct {  
    int data[QUEUE_SIZE];  
    int front;  
    int rear;  
} SeqQueue;
```



# 顺序队列：初始化

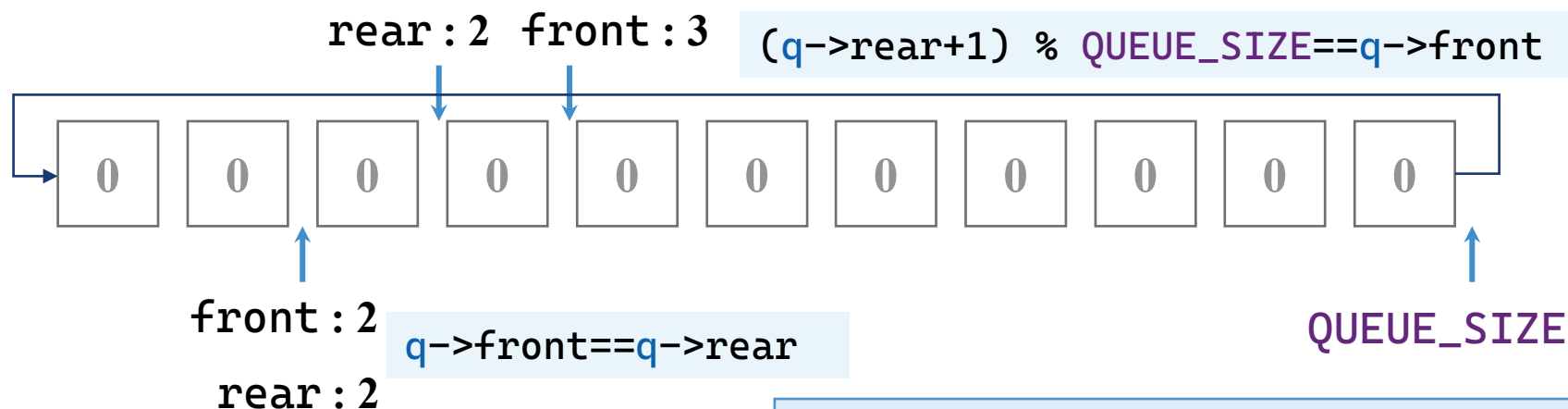
- 初始化：长度清零（头尾皆为0）



```
int init_queue(SeqQueue* q) {  
    if (q == NULL)  
        return ERROR;  
    q->front = 0;  
    q->rear = 0;  
    return OK;  
}
```

# 顺序队列：判断空和满

- 判断空：队列头尾是否相等
- 判断满：队列头是否达到队列尾加一



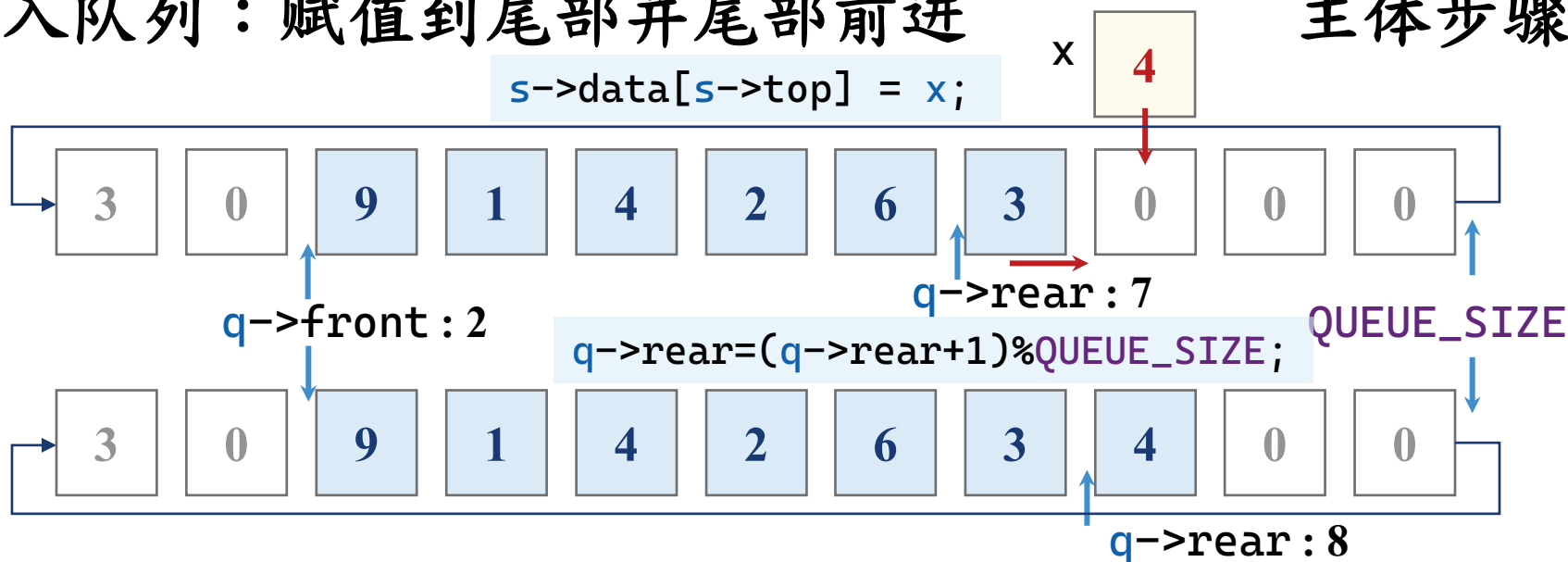
```
int is_empty(SeqQueue* q) {  
    if (q == NULL)  
        return 1;  
    return (q->front == q->rear);  
}
```

```
int is_full(const SeqQueue* q) {  
    if (q == NULL)  
        return 1;    // 防止溢出  
    return ((q->rear + 1) %  
            QUEUE_SIZE == q->front);  
}
```

# 顺序队列：进入队列

- 入队列：赋值到尾部并尾部前进

主体步骤



```
int enqueue(SeqQueue* q, int x) {  
    if (q == NULL || is_full(q))  
        return ERROR;  
    q->data[q->rear] = x;  
    q->rear = (q->rear + 1) % QUEUE_SIZE;  
    return OK;  
}
```

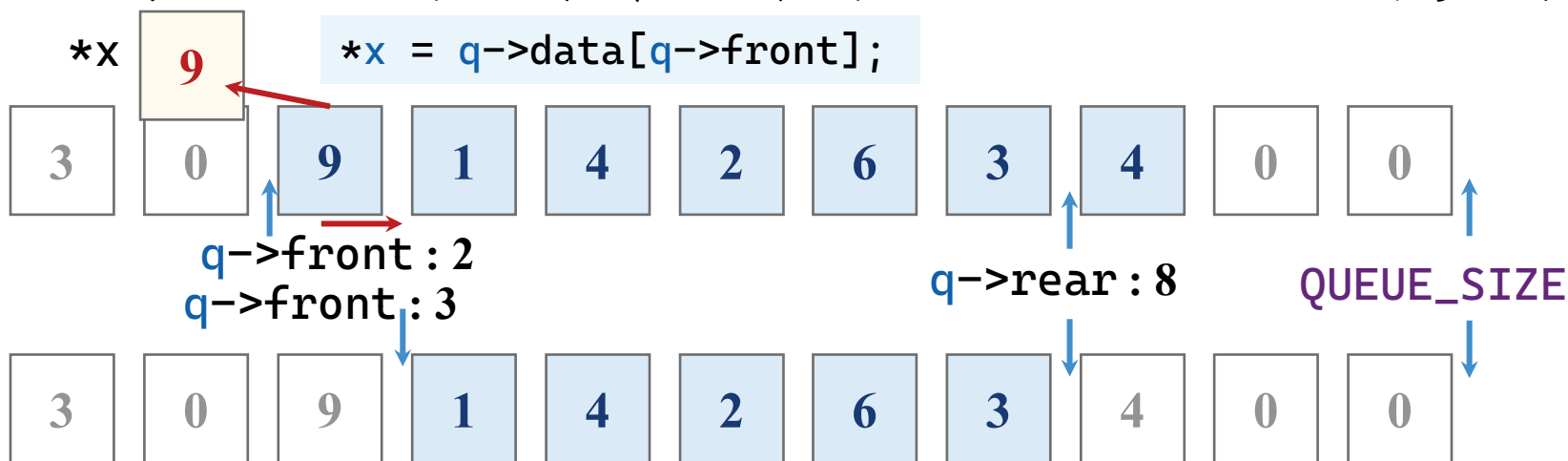
边界判定

1. 队列指针空
2. 队列满

# 顺序队列：弹出队列

- 出队列：赋值到头部并头部前进

主体步骤



```
int dequeue(SeqQueue* q, int* x) {  
    if (q == NULL || is_empty(q) || x == NULL)  
        return ERROR;  
    *x = q->data[q->front];  
    q->front = (q->front + 1) % QUEUE_SIZE;  
    return OK;  
}
```

边界判定

1. 队列指针空
2. 队列空
3. 出指针空

# 目录

|  |     |       |
|--|-----|-------|
|  | 2   | 队列    |
|  | 2.2 | 顺序队列  |
|  | 2.3 | 链队列   |
|  | 2.4 | 队列的应用 |
|  | 3   | 小结    |

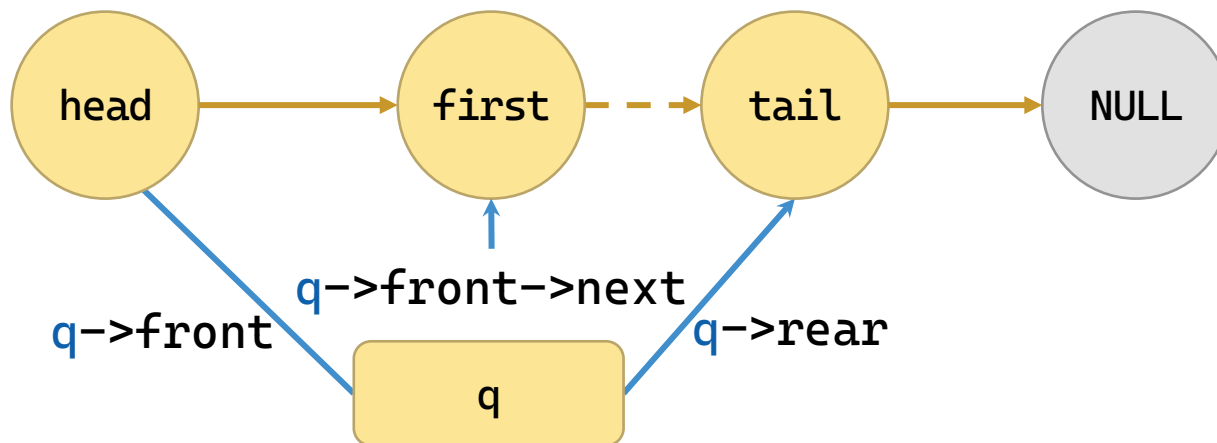
# 链队列的存储结构

- 链队列的存储结构

- 先建立带头结点的链表，避免二级指针，方便操作
- 再将队首尾都设定为插入删除处的前一结点，方便操作

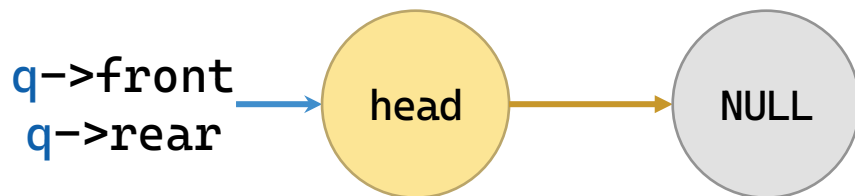
```
typedef struct Node {  
    int data;  
    struct Node* next;  
} LinkNode;
```

```
typedef struct {  
    LinkNode* front; // 队头的前指针  
    LinkNode* rear;  // 队尾指针  
} LinkQueue;
```



# 链队列：初始化

- 初始化：长度清零（顶、尾都为头结点）



```
int init_queue(LinkQueue* q) {  
    if (q == NULL)  
        return ERROR;  
    LinkNode* head = (LinkNode*)malloc(sizeof(LinkNode));  
    if (head == NULL)  
        return ERROR;  
    head->next = NULL;  
    q->front = head;  
    q->rear = head;  
    return OK;  
}
```

# 链队列：判断空

- 判断空：队列头尾是否相等
- 判断满：内部没有办法申请新的空间即是满

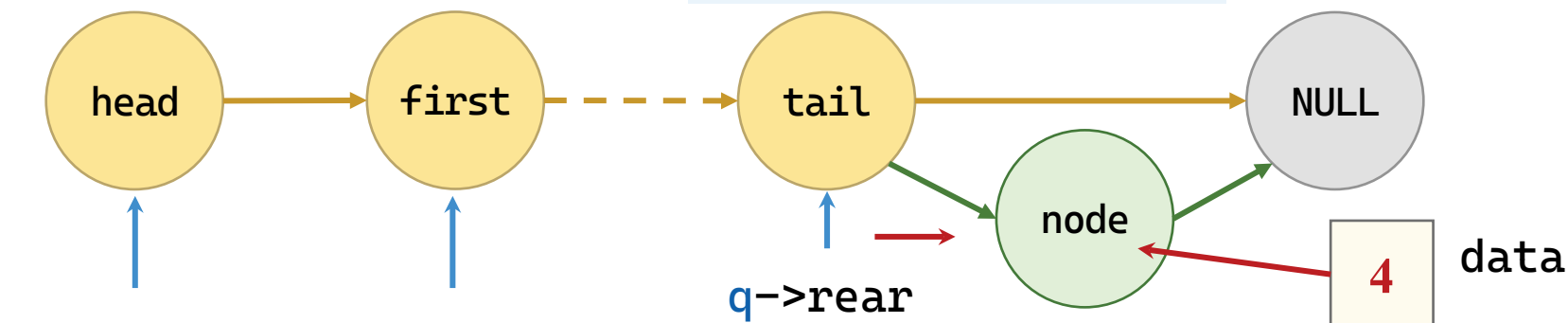
```
int is_empty(const LinkQueue* q) {  
    if (q == NULL) {  
        return 1;  
    }  
    return (q->front == q->rear);  
}
```

# 链队列：入队

## • 入队列：赋值到头部

```
q->rear->next = node;  
q->rear = node;
```

## 主体步骤

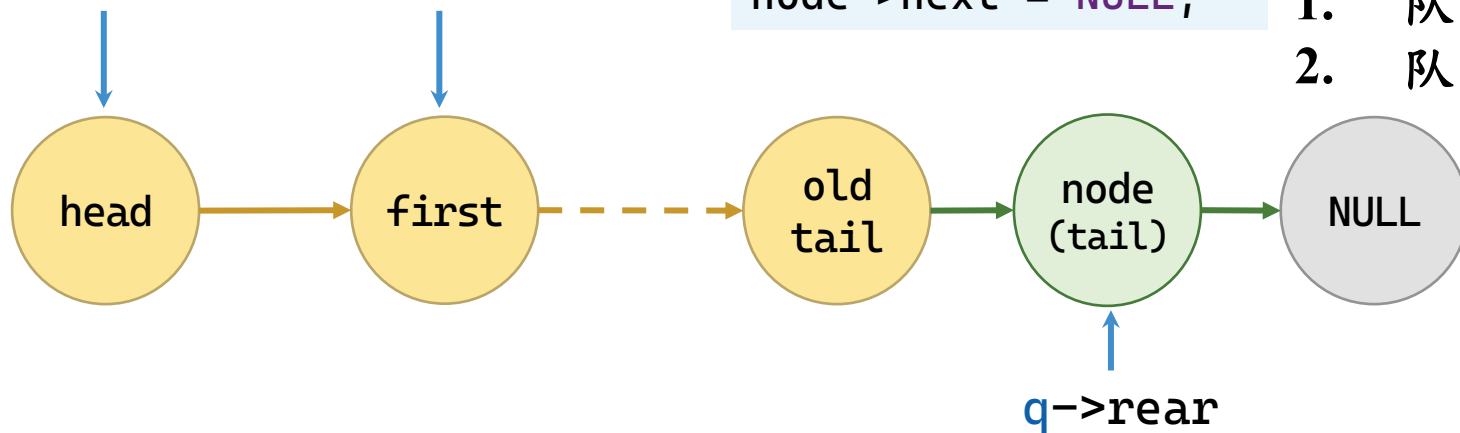


`q->front`   `q->front->next`

```
node->data = x;  
node->next = NULL;
```

## 边界判定

1. 队列指针空
2. 队列满



```

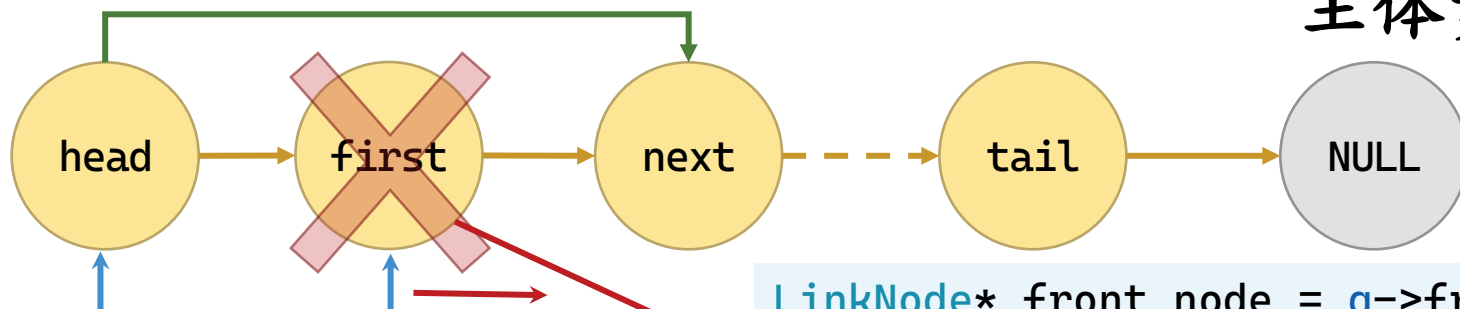
int enqueue(LinkQueue* q, int x) {
    if (q == NULL)
        return ERROR;
    LinkNode* node = (LinkNode*)malloc(sizeof(LinkNode));
    if (node == NULL)
        return ERROR;
    node->data = x;
    node->next = NULL;
    q->rear->next = node;
    q->rear = node;
    return OK;
}

```

# 链队列：出队

- 出队列：自顶部赋值并前进

## 主体步骤

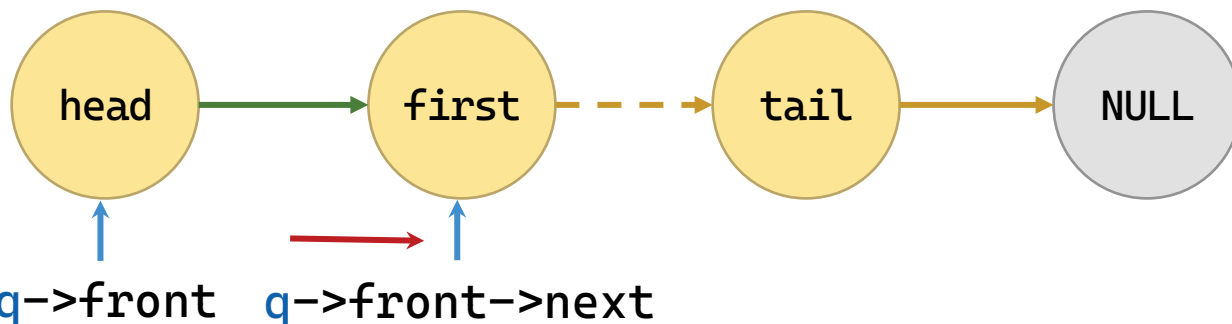


```
front_node = q->front;
first = front_node->next;
*x = first->data;
front_node->next = first->next;
free(first);
```

```
LinkNode* front_node = q->front;
LinkNode* first = front_node->next;
*x = first->data;
```

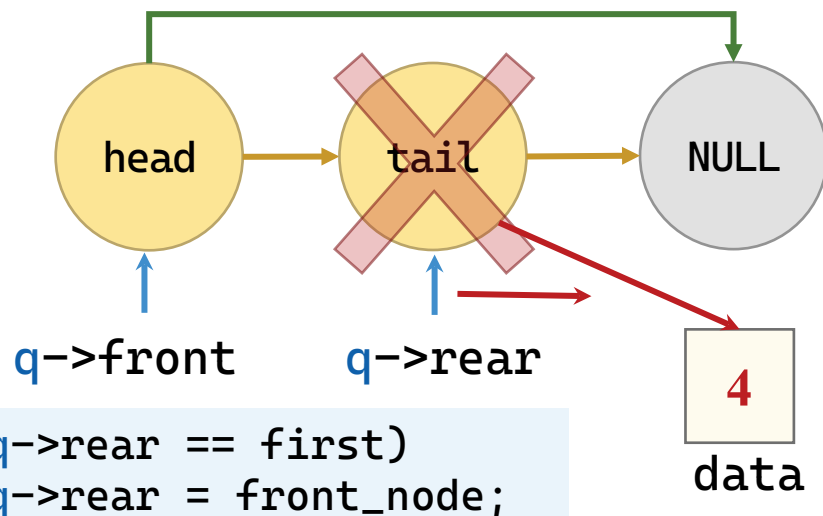
## 边界判定

1. 队列指针空
2. 队列空
3. 出指针空



# 链队列：出队

- 出队列：自顶部赋值并前进



## 特殊情况

当队列只剩一个节点时，  
释放第一节点相当于释放  
尾部，此时头尾节点一致。

```

int dequeue(LinkQueue* q, int* x) {
    if (q == NULL || x == NULL || is_empty(q)) {
        return ERROR;
    }
    LinkNode* front_node = q->front;        // 头结点
    LinkNode* first = front_node->next;      // 第一个数据结点
    *x = first->data;
    front_node->next = first->next;
    if (q->rear == first) {                  // 队列只剩一个数据结点
        q->rear = front_node;
    }
    free(first);
    return OK;
}

```

# 目录

|  |     |       |
|--|-----|-------|
|  | 2   | 队列    |
|  | 2.2 | 顺序队列  |
|  | 2.3 | 链队列   |
|  | 2.4 | 队列的应用 |
|  | 3   | 小结    |

# 应用：腐烂的橘子

- LeetCode 994 ( 腐烂的橘子 ) 在给定的  $m \times n$  网格 grid 中，每个单元格可以有以下三个值之一：
  - 值 0 代表空单元格；
  - 值 1 代表新鲜橘子；
  - 值 2 代表腐烂的橘子。
- 每分钟，腐烂的橘子 周围 4 个方向上相邻 的新鲜橘子都会腐烂。
- 返回 直到单元格中没有新鲜橘子为止所必须经过的最小分钟数。如果不可能，返回 -1 。

# 应用：腐烂的橘子

## • 思路

— 将烂橘子入队，记录鲜橘子个数

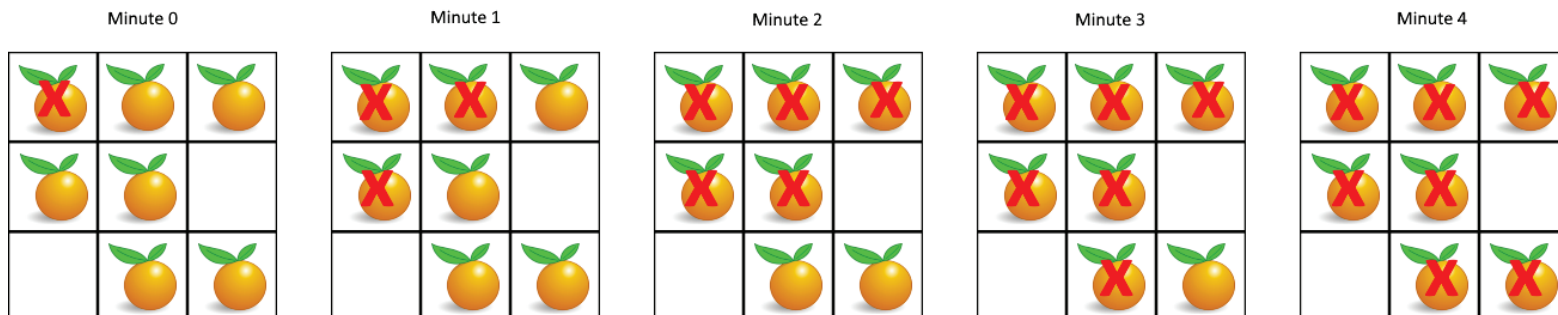
— 依次出队，记录本层长度

▪ 如果是烂橘子，将周围的橘子入队

▪ 时间增加一分钟

— 队列为空则表示传染完毕

▪ 如果已处理了所有鲜橘子，则输出时间减一（有一轮空转）。



```

#include <stdio.h>
typedef struct {
    int row, col;
} Orange;
Orange q[100];
int front = 0, rear = 0;
void enqueue(int r, int c) {
    q[rear].row = r; q[rear++].col = c;
}
Orange dequeue() {
    return q[front++];
}
int isEmpty() {
    return front == rear;
}
int orangesRotting(int** grid, int rows, int cols) {
    front = rear = 0; // 重置队列
    int minutes = 0;
    int freshCnt = 0; // 统计新鲜橘子数量
    for (int i = 0; i < rows; i++) // 1. 传染源入队, 统计新鲜橘子
        for (int j = 0; j < cols; j++) {
            if (grid[i][j] == 2) enqueue(i, j);
            if (grid[i][j] == 1) freshCnt++; // 记录好橘子数量
        }
    if (freshCnt == 0) return 0;
}

```

```

int dir[4][2] = { {-1,0}, {1,0}, {0,-1}, {0,1} };
while (!isEmpty()) {           // 2. BFS 水波纹扩散
    int levelSize = rear - front;
    for (int i = 0; i < levelSize; i++) {
        Orange cur = dequeue();
        for (int d = 0; d < 4; d++) {
            int nr = cur.row + dir[d][0];
            int nc = cur.col + dir[d][1];
            if (nr >= 0 && nr < rows && nc >= 0
                && nc < cols && grid[nr][nc] == 1) {
                grid[nr][nc] = 2; // 变成坏橘子
                freshCnt--;        // 好橘子被传染, 数量减 1
                enqueue(nr, nc);
            }
        }
    }
    minutes++;
}
return (freshCnt == 0) ? (minutes - 1) : -1;
}

int main() {
    int arr[3][3] = { {2, 1, 1}, {1, 1, 0}, {0, 1, 1} };
    int* grid[3] = { arr[0], arr[1], arr[2] };
    printf("结果: %d\n", orangesRotting(grid, 3, 3));
    return 0;
}

```

# 目录

|  |   |    |
|--|---|----|
|  | 1 | 栈  |
|  | 2 | 队列 |
|  | 3 | 小结 |
|  |   |    |
|  |   |    |

# 小结

- 栈：后进先出。
- 队列：先进先出。
- 栈和队列主要用来“保存”中间结果。
  - 输出结果的次序跟中间结果一致的，采用队列保存；
  - 输出结果的次序跟中间结果可能不一致的，采用栈保存。
- 循环队列：正常不应该写一次性队列。
- 递归算法的栈实现。

谢谢观看

理论课程



廈門大學  
XIAMEN UNIVERSITY



信息学院 黄 焯  
(特色化示范性软件学院) 博士, 副教授  
School of Informatics Wei Huang